License: arXiv.org perpetual non-exclusive license
arXiv:2606.23078v1 [eess.IV] 22 Jun 2026

A Systematic Survey on Event Camera Representation Learning

Hongwei Ren* Youxin Jiang* Tuopusen Huang Xiangqian Wu† ††thanks: * equal contribution,†corresponding author.

Abstract

Event cameras offer distinctive advantages, including microsecond-level latency and high dynamic range, rendering them promising for challenging perception tasks. Inspired by biological vision, they output asynchronous and sparse event streams rather than dense image frames, creating a fundamental mismatch with mainstream neural networks. This survey reviews recent advances in event camera representation learning from the perspective of converting raw event streams into learnable representations. We organize existing methods into two main categories: (1) dense-based representations, which transform raw event streams into regular grid-like structures to leverage mature RGB backbones and multimodal fusion pipelines, and (2) sparse-based representations, which retain events as discrete spatio-temporal structures to preserve fine-grained temporal dynamics and data sparsity. This representation-centric organization clarifies how different representations balance structural regularity, temporal fidelity, sparsity preservation, and architectural compatibility. For each category, we examine the underlying design choices, modeling principles, and task-level implications. We further summarize standard benchmarks and evaluation settings across representative high-level perception and low-level vision tasks. Finally, we discuss open problems and outline future research directions toward more efficient, scalable, and robust event-based perception systems.

Index Terms:

Event Camera, Representation Learning.

I Introduction

With the growing demand for computer vision in high-speed and high-dynamic-range (HDR) scenarios, the limitations of traditional frame-based cameras have become increasingly evident. Constrained by fixed sampling rates and limited dynamic range, these sensors often suffer from motion blur, limited exposure adaptability, and substantial data redundancy, making them inadequate for perception in extreme environments *[1]*. In contrast, event cameras, inspired by biological vision *[2]*, offer a fundamentally different sensing paradigm through asynchronous sensing. Instead of capturing full frames at fixed intervals, they respond independently to pixel-wise brightness changes, achieving microsecond-level latency, high dynamic range, sparse event streams, and low power consumption *[3]*. As a result, event cameras have shown strong potential in demanding applications such as autonomous driving, robotics, and high-speed industrial inspection.

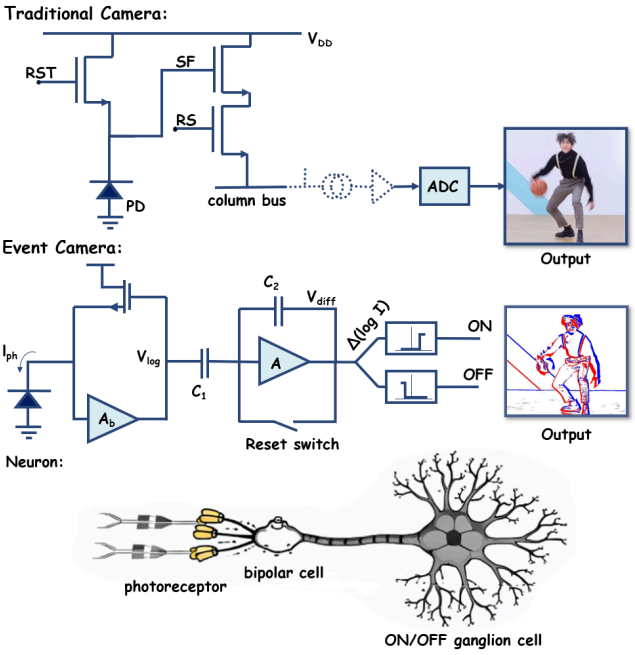


Fig. 1: Comparison between traditional frame-based cameras, event cameras, and biological visual pathways. Event cameras asynchronously generate polarity-aware ON/OFF events in response to local brightness changes, resembling retinal ON/OFF signaling.

From a hardware perspective, event cameras are typically implemented using custom-designed complementary metal- oxide-semiconductor (CMOS) circuits *[3]*, in which each pixel operates independently as an event generator. This pixel-level sensing mechanism is illustrated in Fig. 1. Each pixel contains a photodiode and a logarithmic amplifier that continuously monitors changes in log-intensity. When the brightness change since the last event at that pixel exceeds a predefined contrast threshold, the local circuit is triggered to generate a new event with microsecond-level temporal precision. This process can be formulated as follows:

$$\Delta L(x_k, y_k, t_k) = L(x_k, y_k, t_k) - L(x_k, y_k, t_k^-), \quad (1)$$

$$|\Delta L(x_k, y_k, t_k)| \geq C, \quad p_k = \text{sign}\left(\Delta L(x_k, y_k, t_k)\right), \quad (2)$$

where k denotes the event index, t_k^- denotes the timestamp of the previous event at the same pixel, ΔL is the accumulated log-intensity change, C is the contrast threshold, and $p_k \in \{+1, -1\}$ denotes the event polarity. The resulting output forms a continuous asynchronous event stream, represented as:

$$\mathcal{E} = \{ e_k \}_{k=1}^{N}, \quad e_k = (x_k, y_k, t_k, p_k), \quad (3)$$

where e_k denotes the k-th event, N is the number of events in the stream, and (x_k, y_k), t_k, and p_k represent its spatial location, timestamp, and polarity, respectively.

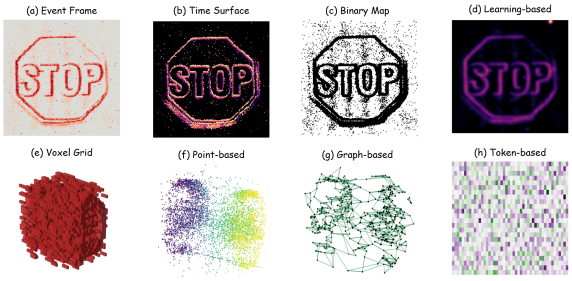


Fig. 2: Overview of diverse event camera representation paradigms using a “STOP” sign scene. (a), (b), (c), (e) show different frame-based mappings; (d) depicts end-to-end learned feature activations;(f) preserves raw sparsity in a 3D temporal point cloud; (g) illustrates graph-based connectivity; (h) represents a structured token latent space.

Although the event stream in Address-Event Representation (AER) form preserves fine-grained temporal information, it is fundamentally mismatched with mainstream neural networks. Unlike standard image data, event streams are spatially sparse, temporally asynchronous, and structurally irregular. Consequently, how to transform raw event streams into representations compatible with modern neural architectures has become a central challenge in event-based vision *[4]*. To address this mismatch, a variety of event representations have been proposed. From the perspective of structural regularity, these representations can be broadly grouped into two paradigms: dense-based representations and sparse-based representations, as illustrated in Fig. 2. Dense-based representations regularize event streams into structured formats for compatibility with conventional architectures, whereas sparse-based representations preserve the discrete and asynchronous nature of events for more faithful event-native modeling.

1) Dense-based representations: Dense-based representations convert raw event streams into regular grid-like structures, such as maps, tensors, or spatio-temporal volumes. The main motivation of this paradigm is to improve compatibility with mature deep learning architectures originally developed for dense visual inputs, such as CNNs and Transformers. By regularizing the event stream into structured arrays, these representations facilitate both feature extraction and multimodal fusion with frame-based modalities. However, such regularization inevitably introduces temporal quantization and information aggregation, which may weaken the native sparsity and fine-grained temporal precision of event data.

a) Map-based representations: This is one of the most widely used dense formulations. It converts the asynchronous event stream within a temporal window into a structured map representation through an aggregation function:

$$\mathcal{M} = \Phi(\mathcal{E}_{[t_s, t_e]}), \quad \mathcal{M} \in \mathbb{R}^{H \times W \times C}, \quad (4)$$

where $\mathcal{M}$ denotes the resulting map-based representation, $\mathcal{E}_{[t_s, t_e]}$ denotes the set of events accumulated within the time interval $[t_s, t_e]$, t_s and t_e are the start and end timestamps of the interval, $\Phi(\cdot)$ denotes the event aggregation operator, and H, W, and C denote the height, width, and channel number of the resulting map, respectively. Although structurally similar to conventional image tensors, this representation encodes temporal information through the aggregation process. Typical examples, as shown in Fig. 2(a)–(c), include event frames, time surfaces, and binary maps.

b) Voxel-grid representations: As an extension of map-based encoding to the spatio-temporal domain, voxel-grid representation is one of the most widely used dense event formulations. Given an event stream within a temporal window, voxel-grid representations organize events into a structured spatio-temporal tensor:

$$\mathcal{V} = \{ V_b \}_{b=1}^{B}, \quad \mathcal{V} \in \mathbb{R}^{B \times H \times W \times C}, \quad (5)$$

where B denotes the number of temporal bins, H and W denote the height and width of the spatial grid, and C denotes the number of channels. Each slice V_b aggregates the events assigned to the b-th temporal bin based on predefined accumulation rules, such as event counting, polarity summation, or interpolated voting. In this way, voxel-grid representations preserve temporal ordering more effectively than single-map encodings, while remaining compatible with standard dense neural architectures.

c) Learned dense representations: Instead of relying on manually designed accumulation rules, some methods learn dense feature representations from raw event streams through parameterized neural mappings:

$$\mathcal{Z} = f_\theta(\mathcal{E}), \quad \mathcal{Z} \in \mathbb{R}^{T \times H \times W \times C}, \quad (6)$$

where $\mathcal{E}$ denotes the input event stream, $f_\theta(\cdot)$ denotes a learnable encoder parameterized by θ, and $\mathcal{Z}$ denotes the resulting dense latent representation. Here, T, H, W, and C denote the temporal dimension, height, width, and channel number of the latent representation, respectively. In these approaches, the representation is optimized end-to-end under task supervision, allowing the network to produce compact and task-adaptive features. Compared with handcrafted encodings, such learned dense representations often provide stronger flexibility and better integration with downstream architectures.

2) Sparse-based representations: In contrast to dense-based methods, sparse-based representations aim to preserve the native form of event streams. Instead of imposing regular grids, these methods maintain the discrete, asynchronous, and irregular characteristics of event data, making them better suited for modeling fine-grained temporal dynamics and event-level interactions. Their key advantage lies in preserving temporal fidelity and data sparsity. However, because standard dense operators are not directly applicable, sparse-based methods usually require more specialized architectures and more complex neighborhood construction strategies.

a) Token-based representations: To better exploit Transformer architectures, some methods decompose event observations or local event regions into token sequences:

$$\mathcal{T} = [\tau_1, \tau_2, ..., \tau_L], \quad \mathcal{T} \in \mathbb{R}^{L \times D}, \quad (7)$$

where τ_i denotes the i-th event token, L denotes the sequence length, and D denotes the embedding dimension. This formulation enables global self-attention to model long-range spatiotemporal dependencies. Although token sequences are structurally regular, their tokens are usually generated from sparse event subsets rather than dense image-plane grids, and are therefore categorized as sparse-based representations in this survey.

b) Graph-based representations: Graph-based representations are a representative sparse event formulation. They represent the event stream as a graph:

$$\mathcal{G} = (\mathcal{N}, \mathcal{A}, \mathbf{X}), \qquad \mathcal{A} = \{ (n_i, n_j) | \phi(n_i, n_j) = 1 \}, \quad (8)$$

where $\mathcal{N} = \{ n_i \}_{i=1}^{M}$ denotes the node set, M denotes the number of nodes, $\mathcal{A}$ denotes the edge set, and $\mathbf{X} \in \mathbb{R}^{M \times D}$ is the node feature matrix. The connectivity function $\phi(\cdot,\cdot)$ determines whether two nodes are connected according to spatial, temporal, or semantic affinity. Each node may correspond to an individual event or a local spatiotemporal region. By organizing event data into a non-Euclidean relational structure, graph-based representations are well-suited for modeling the irregular and asynchronous nature of event streams, enabling graph neural networks to capture both local interactions and long-range dependencies.

c) Point-based representations: Point-based representations are the closest to the raw event stream. They represent events as a set of discrete spatio-temporal points:

$$\mathcal{P} = \{ \mathbf{q}_i \}_{i=1}^{N}, \quad \mathbf{q}_i = (x_i, y_i, t_i, p_i^*), \quad (9)$$

where $\mathbf{q}_i$ denotes the i-th event point, N denotes the number of events, (x_i, y_i) and t_i denote its spatial location and timestamp, and p_i^* denotes an optional polarity attribute when polarity information is used. As shown in Fig. 2(e), point-based representations preserve the original spatio-temporal distribution of events without explicit frame conversion, making them suitable for event-native modeling.

Summary: Overall, the above taxonomy reveals that existing event representations mainly differ in whether they regularize events into dense structures or preserve their sparse and asynchronous nature. Dense-based representations are generally more compatible with mature backbones and multimodal pipelines, while sparse-based representations better maintain temporal fidelity and native sparsity. The evolution of event representation learning can therefore be understood as an effort to balance architectural compatibility, computational efficiency, and representation fidelity.

Figure 3. Taxonomy of event camera representation learning. Existing methods are grouped into dense-based and sparse-based paradigms according to their structural regularity and event sparsity.

II Review Methodology

In preparing this survey, we adopted a systematic literature collection and screening process to ensure broad coverage and technical relevance. Relevant papers were gathered from major academic databases, including IEEE Xplore, ScienceDirect, SpringerLink, arXiv, and Google Scholar, using combinations of keywords such as “event camera”, “event-based vision”, “event representation”, “representation learning”, “event frame”, “voxel grid”, “graph”, “point cloud”, and “transformer”, together with downstream task terms including “classification”, “detection”, “segmentation”, “optical flow”, and “reconstruction”. We mainly included studies that explicitly focused on the design, learning, or application of event representations and provided clear methodological descriptions and experimental validation, while excluding works with limited relevance or substantial overlap. Based on this process, we selected and reviewed over 120 representative papers. These papers were further organized into dense-based and sparse-based paradigms, together with additional references on event camera fundamentals, benchmark datasets, and representative downstream tasks.

III Differences from Existing Surveys

In this section, we highlight the differences between our survey and existing reviews in order to clarify the significance of our work. Through a careful examination of the current literature, we find that existing surveys on event-based vision mainly focus on broad overviews of sensor technology, general event-driven vision pipelines, specific application domains, or benchmark-oriented summaries of deep learning methods, while relatively few works provide a dedicated and systematic review of event camera representation learning itself. The survey in *[5]* mainly emphasizes recent advances in event camera hardware, sensing mechanisms, and emerging applications, with limited discussion of how raw event streams are transformed into learnable representations. The survey in *[6]* presents a broad overview of event-driven sensing, foundational principles, and representative tasks, but it does not establish a focused taxonomy centered on representation modeling. The survey in *[7]* concentrates on automotive perception scenarios, highlighting the role of event cameras in driving-related applications, yet its discussion is domain-specific rather than representation-oriented. The survey in *[8]* reviews deep learning frameworks, benchmark datasets, and evaluation settings for event-based vision tasks, but its primary emphasis lies in model performance and benchmark coverage rather than in systematically analyzing the structural design space of event representations.

In contrast to these works, our survey specifically focuses on event camera representation learning, namely, how raw asynchronous event streams are transformed into learnable structures that can be effectively processed by modern neural architectures. In particular, we organize existing methods into two major paradigms, dense-based and sparse-based representations, and further analyze their structural characteristics, design principles, modeling trade-offs, and task-level implications. Beyond reviewing representative methods, we also summarize benchmark datasets and evaluation settings across major task families, covering representative high-level perception and low-level vision tasks, while discussing open challenges and future directions in event representation learning. Therefore, compared with existing surveys, our work provides a more focused, representation-centric, and structurally unified perspective on the development of event-based vision.

IV Dense-based Representation

Dense-based representations transform asynchronous event streams into regular grid-like structures, such as 2D maps, multi-channel tensors, or 3D spatio-temporal volumes. Given an event stream $\mathcal{E} = \{ e_i \}_{i=1}^{N}$, where $e_i = (x_i, y_i, t_i, p_i)$ denotes the i-th event, dense methods typically project irregular events onto structured arrays through temporal accumulation, timestamp overwriting, binning, or learned aggregation. This conversion makes event data directly compatible with mature vision backbones, including CNNs, U-Nets, recurrent networks, and Transformers. The price of this compatibility is that part of the microsecond-level temporal precision and native sparsity of Address-Event Representation (AER) is often traded for spatial continuity, batch processing efficiency, and architectural convenience.

IV-A Event Frame

The Event Frame is the most intuitive dense representation, treating the events occurring within a temporal interval as an image-like observation. It accumulates sparse events onto a two-dimensional spatial grid and therefore establishes a direct bridge between event cameras and conventional frame-based visual processing. Compared with raw event streams in AER form, event frames discard the exact ordering of events inside the integration window, but they provide a compact structure that can be processed by standard image-based vision networks.

Formally, given a temporal window $\Delta T = [t_s, t_e]$, an event frame can be constructed by accumulating events at their corresponding pixel locations. Since positive and negative events describe opposite brightness changes, most practical implementations adopt polarity-separated channels:

$$H_{\pm}(x,y) = \sum_{e_i \in \mathcal{E},\, t_i \in \Delta T,\, p_i = \pm 1} \delta(x - x_i, y - y_i), \quad (10)$$

where $\delta(\cdot)$ denotes the Kronecker delta function, and H_+ and H_- record the numbers of positive and negative events, respectively. This formulation is closely related to early event histograms and pseudo-pictures [9, 10], which established the basic principle of converting asynchronous event streams into image-plane accumulations through polarity-aware counting. Owing to their simplicity and architectural compatibility, event-frame representations have been widely adopted in representative event-based pipelines, including early frame-style mappings [9], SNN-based recognition [11, 12], object tracking [13, 14], semantic segmentation [15], optical flow estimation [16], and depth estimation [17].

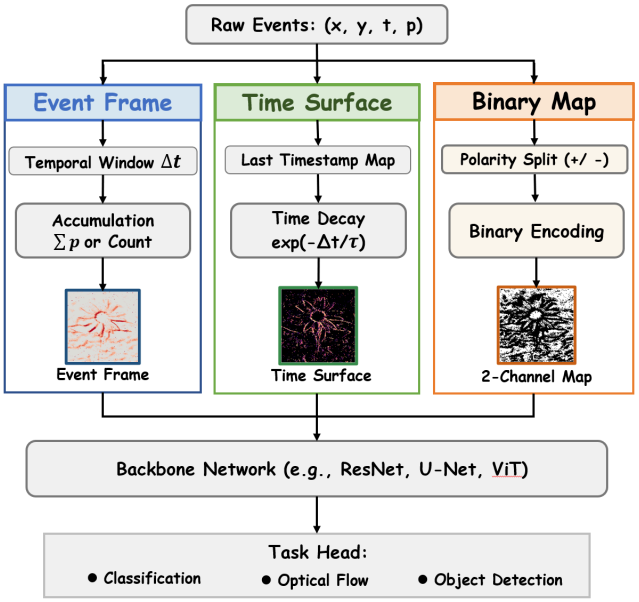


Fig. 4: Overview of representative map-based representations. Raw events are projected into grid-structured formats, including Event Frame, Time Surface, and Binary Map, which emphasize event accumulation, temporal recency, and binary occupancy encoding, respectively.

The representational behavior of event frames is mainly determined by the accumulation strategy. Fixed-time accumulation generates one frame within each constant temporal duration ΔT [3, 18, 19]. This strategy is simple, deterministic, and naturally aligned with frame-based processing pipelines, but the resulting event density varies significantly with scene dynamics: static scenes may produce extremely sparse frames, whereas fast motion may lead to saturated contours and edge smearing. Fixed-event-number accumulation instead generates a frame after receiving a predefined number of events [20, 3]. By adapting the temporal length of the window to the event rate, this strategy produces more stable frame density under changing motion conditions, but it also makes the temporal duration of each frame scene-dependent.

Another important refinement is motion-aware accumulation. Instead of directly projecting all events in the window onto the same spatial plane, motion-compensated event frames align events to a reference timestamp before accumulation [21]. This design reveals that the limitation of event frames is not only caused by their two-dimensional grid structure, but also by whether the aggregation process respects the underlying motion. When the motion model is accurate, motion compensation can produce sharper spatial structures; when it is unavailable or inaccurate, temporally distinct observations are still collapsed into the same static map.

Overall, event frames provide a simple and efficient bridge between asynchronous events and image-based neural architectures. Their main advantage lies in low memory cost, easy implementation, and strong compatibility with existing vision backbones. However, their temporal aggregation discards fine-grained event ordering, making them less suitable for tasks that require precise temporal reasoning.

IV-B Time Surface

Different from event frames that emphasize event frequency, the time surface represents event streams through temporal recency. It records how recently each pixel has fired and converts asynchronous event streams into a two-dimensional map whose intensity reflects the freshness of local activity. This design preserves a stronger temporal cue than simple counting, because the value at each pixel is determined by the latest timestamp rather than the total number of accumulated events. Therefore, time surfaces can be viewed as recency-weighted dense memories of event streams.

The standard formulation follows the Surface of Active Events [22]. For a pixel (x, y), let $t_{\text{last}}(x, y)$ be the timestamp of the most recent event at that location before the current time t. The time surface is defined as:

$$S(x,y,t) = \exp\left(-\frac{t - t_{\text{last}}(x,y)}{\tau}\right), \quad (11)$$

where τ is the decay constant controlling how quickly past activity fades. A small τ emphasizes only very recent events, while a large τ preserves longer motion traces. In this sense, the key design variable of time surfaces is not event count, but the decay rule that determines how temporal history is retained and forgotten.

Several variants further refine the temporal memory mechanism of time surfaces. HOTS [23] extends pixel-wise recency maps into a hierarchical representation, showing that local time-surface patterns can be progressively organized into higher-level spatio-temporal descriptors. Specifically, for an incoming event $e_i = (x_i, y_i, t_i, p_i)$, HOTS extracts a local time-surface patch around its spatial location:

$$s_i(\Delta x, \Delta y) = \exp\left(-\frac{t_i - t_{\text{last}}(x_i + \Delta x, y_i + \Delta y)}{\tau}\right), \quad (12)$$

$$\mathbf{s}_i = \{\, s_i(\Delta x, \Delta y) | (\Delta x, \Delta y) \in \Omega \,\}. \quad (13)$$

where Ω denotes the local neighborhood centered at (x_i, y_i), $(\Delta x, \Delta y)$ indexes the relative spatial offsets within this neighborhood, and $\mathbf{s}_i$ represents the recent spatio-temporal activity pattern around the current event. HOTS learns hierarchical prototypes from such local time-surface patches, whereas HATS *[24]* aggregates them within spatial cells to obtain more stable local descriptors. Speed-invariant time surfaces *[25]* and Distance Surface *[26]* further reduce the dependence of recency values on motion speed by modifying the decay rule or replacing timestamp decay with distance to recent event structures. More recent adaptive and motion-aware designs *[27, 28, 29]* adjust the decay process or encode motion cues to maintain stable temporal traces under varying event rates and spatial displacements. These developments suggest that the key challenge of time-surface representation is to preserve meaningful short-term motion memory while avoiding stale, speed-sensitive, or blurred structures.

Overall, time surfaces encode event streams through temporal recency rather than event counts, providing a compact description of short-term motion activity. They preserve more temporal information than simple event frames and are effective for local motion and feature patterns. However, because only the most recent timestamp is retained at each pixel, they remain limited in modeling long-range temporal dependencies.

IV-C Binary Map

The Binary Map representation encodes the presence or absence of events at each pixel within a temporal interval. Unlike event frames that accumulate event counts, binary maps focus on occupancy. A pixel is activated once at least one event occurs within the integration window. A unified polarity-separated formulation can be written as:

$$b_k^p(x,y) = \mathbb{I}\left[\exists\, e_i \in \mathcal{E} : (x_i, y_i) = (x,y), t_i \in \tau_k, p_i = p\right], \quad (14)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function, τ_k denotes the k-th temporal sub-window of the full interval ΔT, $k = 1, \ldots, K$, and $p \in \{+1, -1\}$. When $K = 1$, this formulation reduces to a standard binary event map over the full interval ΔT; when $K > 1$, it produces an ordered sequence of binary occupancy maps.

Temporal Binary Representation (TBR) *[30]* introduced this bit-packing strategy by converting a sequence of binary event maps into a compact integer-valued representation:

$$B^p(x,y) = \sum_{k=1}^{K} b_k^p(x,y) \cdot 2^{k-1}. \quad (15)$$

Assuming $\tau_1, \ldots, \tau_K$ are ordered from early to late, different bits indicate whether a pixel is activated in different temporal sub-windows, allowing the representation to preserve coarse temporal order without storing a full temporal volume. Later, Bina-Rep *[31]* emphasized the effectiveness of simple binary event frames, showing that occupancy-based maps can provide a compact alternative to count-based accumulation. Together, these works indicate that binary event representations trade event counts and precise timestamps for efficient occupancy codes: TBR focuses on compact temporal ordering through bit packing, whereas Bina-Rep highlights the simplicity of binary activation maps.

Overall, binary maps provide compact occupancy-based event representations with low memory and computation costs. They are attractive for efficient recognition and hardware-friendly processing. However, their binary encoding discards event multiplicity, precise timing, and richer polarity interactions, limiting their representational capacity.

IV-D Voxel Grid

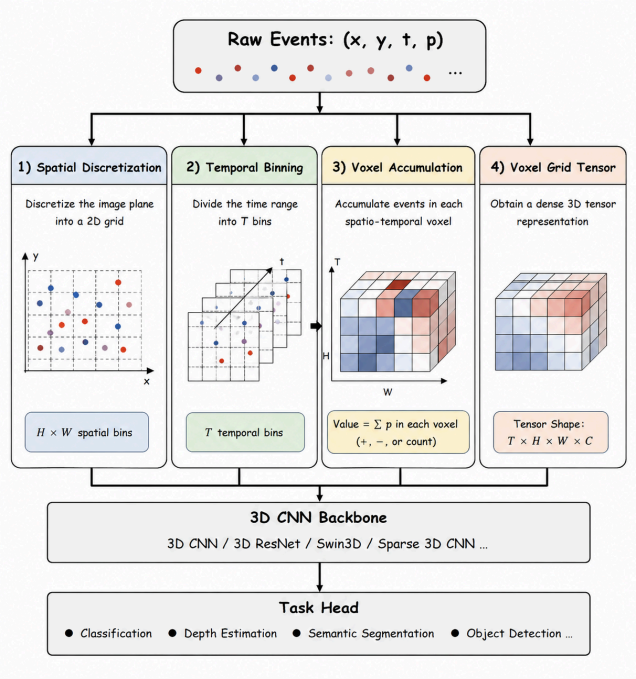


Fig. 5: Overview of voxel-grid event representation. Raw events are discretized along spatial and temporal dimensions, accumulated into spatio-temporal voxels, and organized as a dense 3D tensor for downstream dense neural backbones and task heads.

The voxel-grid representation is one of the most widely used dense spatio-temporal representations for event cameras. It organizes an event stream into a regular 3D volume, where the spatial axes correspond to image coordinates and the temporal axis is divided into multiple discrete bins. Unlike event frames that collapse all events within a time window into a single 2D map, voxel grids preserve a coarse temporal order by accumulating events into temporally ordered slices. By discretizing events along the temporal dimension, voxel grids transform asynchronous event streams into regular spatio-temporal tensors that can be processed by dense visual architectures.

Formally, given an event window $\mathcal{E}_{[t_s,t_e]}$, where t_s and t_e denote the start and end timestamps of the event window, respectively, the timestamp of each event is first normalized to a continuous bin coordinate:

$$\tau_i = (B-1)\frac{t_i - t_s}{t_e - t_s}, \quad (16)$$

where B denotes the number of temporal bins, and τ_i represents the normalized temporal position of event e_i within the voxel grid. A standard voxel grid then accumulates events into a tensor $\mathbf{V} \in \mathbb{R}^{B\times H\times W}$ by:

$$\mathbf{V}_{b,x,y} = \sum_{e_i \in \mathcal{E}_{[t_s,t_e]}} p_i \delta(x - x_i)\delta(y - y_i)\kappa(b - \tau_i), \quad (17)$$

where $\mathbf{V}_{b,x,y}$ denotes the accumulated event value at temporal bin b and spatial location (x, y), H and W are the height and width of the sensor resolution, and p_i is the polarity of event e_i. The spatial delta functions indicate that the event contributes only to its corresponding pixel location, while $\kappa(b - \tau_i)$ determines how the event is assigned to temporal bins according to its normalized timestamp. In practice, $\kappa(\cdot)$ can be implemented as a hard assignment kernel or a differentiable triangular interpolation kernel. Therefore, the construction of voxel grids is mainly determined by the temporal window size, the number of bins B, and the temporal assignment kernel $\kappa(\cdot)$.

The early development of voxel-grid representations was largely driven by dense motion estimation and event-based reconstruction. Differentiable voxel accumulation made it possible to connect asynchronous events with dense prediction networks *[32]*, while event volumes further demonstrated their effectiveness for high-speed and high-dynamic-range video reconstruction *[33]*. These works explain why voxel grids became a default representation for low-level event vision: they retain coarse temporal variation useful for motion reasoning while preserving the regular spatial layout required by dense image restoration and prediction networks.

Subsequent studies extended voxel-grid representations beyond basic dense accumulation by combining them with stronger temporal modeling architectures. Multi-scale, Transformer-based, recurrent, sparse, and state-space designs further exploit the ordered voxel structure to capture spatio-temporal dependencies across bins and spatial locations *[34, 35, 36, 37, 38, 39]*. In this sense, voxel grids have evolved from a simple event-to-volume encoding into a general dense interface between event streams and modern neural architectures. Their regular image-plane alignment also makes them especially suitable for dense prediction and multimodal fusion, where outputs must remain spatially aligned with image coordinates *[40, 41, 42, 43]*.

Overall, voxel grids convert event streams into dense spatio-temporal tensors that preserve coarse temporal order while remaining compatible with mature neural architectures. Their advantage lies in structural compatibility: they retain spatial topology, support temporal modeling across bins, and can reuse dense backbones developed for image and video processing. However, temporal binning still loses microsecond-level ordering, memory cost increases with $B \times H \times W$, and fixed temporal windows may be suboptimal under highly variable event rates. Future voxel-grid research therefore requires more adaptive binning strategies, sparse or compressed voxel processing, and hybrid integration with token-, graph-, or point-based representations.

IV-E Learned Dense Representation

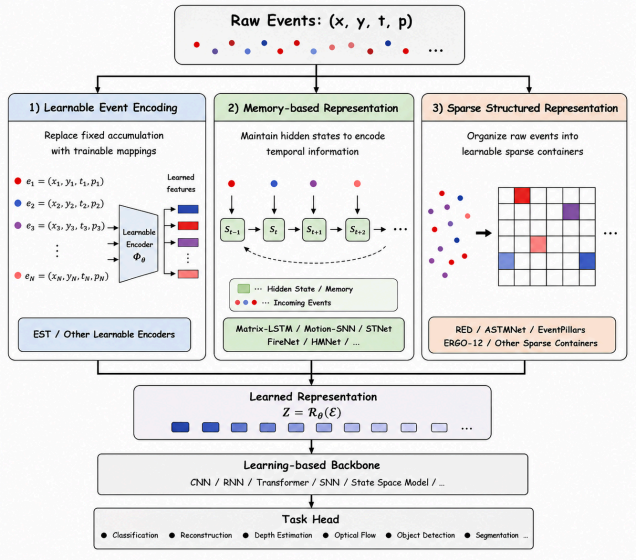


Fig. 6: Illustration of learned dense event representation. Raw events are mapped into learnable representations through event-wise trainable encoding, memory-based temporal state modeling, and sparse structured organization, which are then processed by learning-based backbones for downstream tasks.

The learned dense representation treats the mapping from raw events to feature representations as a learnable component rather than a fixed preprocessing step. Unlike handcrafted representations, such as event frames, time surfaces, binary maps, and voxel grids, which rely on predefined accumulation, decay, thresholding, or temporal discretization rules, learning-based representations directly learn how raw asynchronous events should be projected, aggregated, and stored according to data distribution and task supervision. By integrating representation construction into the optimization process, these methods adapt temporal integration, polarity handling, spatial aggregation, and memory update mechanisms to downstream objectives.

Formally, given an event stream $\mathcal{E} = \{e_i\}_{i=1}^{N}$, where $e_i = (x_i, y_i, t_i, p_i)$, a general learning-based representation can be expressed as:

$$\mathbf{Z} = \mathcal{R}_\theta(\mathcal{E}) = \mathrm{Agg}_{i=1}^{N} \Phi_\theta(x_i, y_i, t_i, p_i, \mathbf{s}_i), \quad (18)$$

where $\mathbf{Z}$ denotes the learned representation, $\mathcal{R}_\theta$ is a learnable event-to-representation mapping, $\Phi_\theta(\cdot)$ encodes individual events or local event groups into latent features, $\mathbf{s}_i$ denotes an optional memory or contextual state, and $\mathrm{Agg}(\cdot)$ aggregates event-wise features into a structured representation. Different methods mainly differ in the design of the embedding function Φ_θ, the form of the state variable $\mathbf{s}_i$, and the aggregation mechanism $\mathrm{Agg}(\cdot)$.

Early learning-based representations primarily focused on replacing handcrafted accumulation rules with differentiable event encoding. EST [44] replaces fixed temporal binning with learnable basis functions, allowing timestamps to contribute adaptively to feature channels while preserving compatibility with convolutional backbones. Matrix-LSTM [45] further introduces recurrent memory into representation construction, enabling the model to learn when temporal information should be retained, updated, or discarded. These methods demonstrate the transition from manually designed accumulation rules to trainable event encoding.

A second line of research introduces memory-based representations, where temporal information is maintained through learnable hidden states rather than explicit accumulation. Motion-SNN [46] and STNet [47] encode event dynamics through spiking or recurrent states, while FireNet [48] learns recurrent representations for high-speed image reconstruction. HMNet [49] further extends this idea through hierarchical neural memories that continuously integrate sparse event updates over multiple temporal scales. These approaches demonstrate that representation learning can be performed through memory evolution instead of static feature construction.

Recent studies have also explored sparse structured representations that directly organize raw events into learnable sparse containers. RED [50] and ASTMNet [51] learn task-specific representations from high-rate event streams without handcrafted accumulation. EventPillars [52] and ERGO-12 [53] further construct learnable sparse structures that preserve event sparsity while supporting efficient downstream perception. These methods reflect a growing trend toward jointly optimizing representation learning, computational efficiency, and task-specific inference.

Overall, learning-based representations replace fixed handcrafted rules with trainable event-to-feature mappings. They can adapt temporal integration, polarity handling, and memory updates to downstream objectives. However, they are often less interpretable, more data-dependent, and computationally heavier than fixed representations.

V Sparse Representation

The sparse representation aims to preserve the inherent sparsity of raw events by processing only informative event locations rather than constructing dense image-like structures. Unlike dense representations that populate regular grids regardless of event occupancy, sparse representations store and propagate features only at activated spatial or spatio-temporal positions, thereby reducing redundant computation and memory consumption. Depending on how sparsity is organized and exploited, existing approaches may represent events as sparse tensors, irregular point sets, graph-structured entities, or compact token sequences. Although these formulations differ in their underlying data structures and neighborhood definitions, they share a common objective: maintaining the efficiency and fine-grained temporal characteristics of asynchronous events while enabling effective feature extraction and long-range dependency modeling.

V-A Token-based Representation

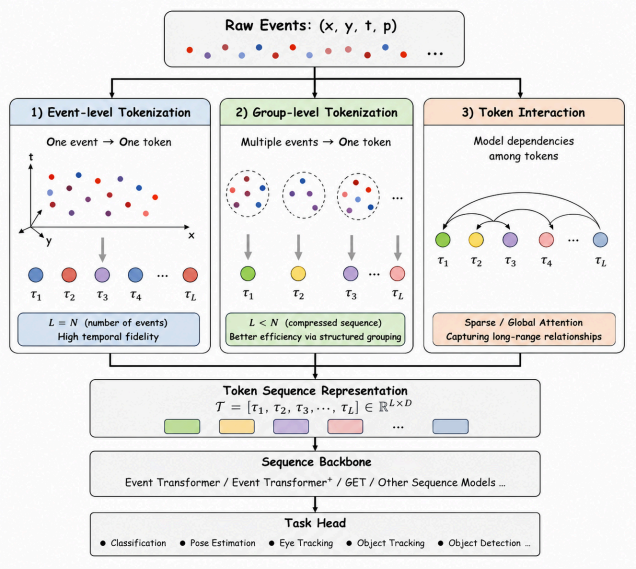


Fig. 7: Illustration of token-based event representation. Raw events are organized into token sequences either by assigning each event to an individual token or by grouping multiple events into compact tokens, followed by token interaction modeling through sparse or global attention for downstream perception tasks.

The token-based representation is a sequence-oriented event encoding paradigm that organizes raw event streams into data-dependent token sequences for sequence modeling. Unlike dense representations that rely on fixed grid structures, token-based representations abstract asynchronous events into variable-length token units directly from raw events without pre-generated event frames, voxel grids, or other handcrafted encodings. Depending on the tokenization strategy, each token may correspond to an individual event or a local event group. Tokenization, therefore, determines which event subsets are treated as atomic units for subsequent sequence interactions.

Formally, given an event stream $\mathcal{E} = \{ e_i \}_{i=1}^{N}$, where each event is represented as $e_i = (x_i, y_i, t_i, p_i)$, a token-based representation constructs a token sequence:

$$\mathcal{T} = [\tau_1, \tau_2, ..., \tau_L] \in \mathbb{R}^{L \times D}, \quad (19)$$

where $\mathcal{T}$ denotes the resulting token representation, L is the number of tokens, and D is the embedding dimension of each token. The value of L depends on the tokenization strategy: event-level tokenization assigns one token to one event, whereas group-level tokenization aggregates multiple events into compact token units to improve efficiency. A general tokenization process can be expressed as:

$$\tau_k = \mathcal{A}(\{ \mathrm{Emb}_\theta(e_i) | e_i \in \mathcal{G}_k \}) + \mathrm{PE}(x_k, y_k, t_k, p_k), \quad (20)$$

where $\mathcal{G}_k \subseteq \mathcal{E}$ denotes the subset of events assigned to the k-th token, $\mathrm{Emb}\,\theta(\cdot)$ maps raw event attributes into a latent feature space, $\mathcal{A}(\cdot)$ aggregates embedded event features within $\mathcal{G}_k$, and $\mathrm{PE}(\cdot)$ injects positional and polarity information derived from the associated event subset. This formulation reveals that token-based representations mainly differ in how event subsets are constructed, how local event information is aggregated, and how positional

information is encoded. In this survey, we specifically restrict token-based representations to methods that generate tokens directly from raw event streams rather than from pre-generated dense representations, since tokenization in the latter primarily serves as a Transformer input format instead of a representation construction mechanism.

Early token-based studies primarily focused on enabling efficient attention over sparse event streams. Event Transformer *[54]* partitions asynchronous events into local event sets and performs sparse-aware attention directly on the resulting event tokens, substantially reducing the computational burden of global attention while preserving long-range interactions. Event Transformer^{+} *[55]* further generalizes this paradigm into a unified framework for event data processing, demonstrating that raw-event tokenization can support diverse event-based tasks within a common Transformer architecture.

Subsequent studies shifted the focus from efficient token interaction to token construction itself. GET *[56]* organizes asynchronous events into group tokens according to temporal and polarity cues, and employs Event Dual Self-Attention together with Group Token Aggregation to facilitate information exchange among complementary token groups. This design demonstrates that tokenization can serve as a task-aware compression mechanism rather than a passive format conversion step.

More recently, event-level tokenization has emerged to maximize temporal fidelity. Jiang et al. *[57]* directly convert raw events into event tokens without temporal discretization and employ Transformer-based interactions to jointly model local and global dependencies. By preserving the original event granularity, this design maintains microsecond-level temporal cues at the cost of substantially longer token sequences.

Overall, token-based representations organize raw events or local event groups into compact embedding sequences for attention-based modeling. They provide a flexible way to capture long-range dependencies while reducing the need for dense grid construction. However, event-level tokens can be computationally expensive, and group-level tokens may lose fine-grained temporal details.

V-B Graph-based Representation

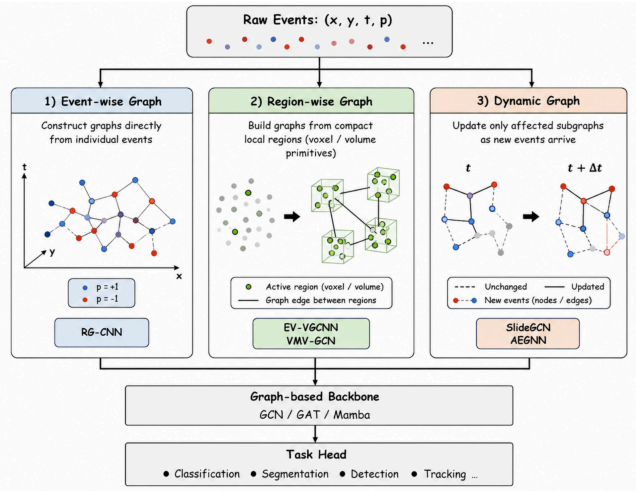


Fig. 8: Illustration of graph-based event representation. Raw events are represented as graphs constructed either from individual events, compact local regions, or dynamically updated subgraphs, enabling graph-based backbones to capture spatial, temporal, and relational dependencies for downstream perception tasks.

The graph-based representation describes an event stream as a non-Euclidean relational structure. Since events are naturally sparse, asynchronous, and distributed along motion trajectories, representing them on regular grids may obscure their underlying topology and introduce redundant computation over empty regions. Instead of relying on fixed spatial layouts, graph-based representations explicitly model relational dependencies by constructing nodes and edges over sparse event structures. In this formulation, the central question is not only how to encode individual events, but also how to determine which events should exchange information.

Formally, given an event set $\mathcal{E} = \{ e_i = (x_i, y_i, t_i, p_i) \}_{i=1}^{N}$, a general graph-based representation can be written as

$$\mathcal{G} = (\mathcal{V}, \mathcal{A}, \mathbf{H}), \quad v_j = \mathcal{C}(\mathcal{S}_j), \quad h_j = \psi(v_j), \tag{21}$$

where $\mathcal{V}$ denotes the node set, $\mathcal{A}$ denotes the adjacency set, and $\mathbf{H}$ is the node feature matrix. Here, $\mathcal{S}_j \subseteq \mathcal{E}$ denotes the event subset assigned to node v_j, $\mathcal{C}(\cdot)$ is the node construction operator, and h_j is the node feature generated by $\psi(\cdot)$. The adjacency set $\mathcal{A}$ is typically determined by spatial, temporal, spatio-temporal, or learned feature affinity, which controls how information is propagated among graph nodes.

Early graph-based representations explicitly construct relational structures over sparse event entities. In this event-wise graph formulation, nodes usually correspond to individual events, and edges are established according to spatial, temporal, or spatio-temporal proximity. RG-CNN *[58]* constructs spatio-temporal graphs over event streams and aggregates neighborhood information through graph convolution, demonstrating that event relationships can be modeled through explicit message passing rather than dense spatial layouts. However, event-wise graph construction becomes computationally expensive under high event rates and is sensitive to isolated noisy events.

To improve scalability, subsequent studies construct graphs over compact local regions rather than individual events. In this region-wise formulation, graph nodes are derived from voxelized or volumetric event primitives, and only informative local regions participate in relational reasoning. EV-VGCNN *[59]* constructs graph vertices from voxelized event regions, while VMV-GCN *[60]* introduces volumetric multi-view graph modeling to capture complementary spatio-temporal structures. Although these methods may use voxelized primitives for node construction, they are still categorized as sparse graph representations because message passing is performed over selected nodes and edges rather than over a full dense lattice. This design reduces redundant computation while preserving structured neighborhood information.

A further line of work focuses on dynamic graph updating, which better matches the asynchronous nature of event streams. Instead of reconstructing the entire graph for each event window, SlideGCN *[61]* updates event relations through a sliding graph mechanism, while AEGNN *[62]* updates only the local subgraph affected by newly arrived events. These methods show that graph-based representation depends not only on node and edge definitions, but also on how the relational structure evolves over time. More recently, EventMG *[63]* further explores multilevel Mamba-Graph modeling, suggesting that dynamic graph representations may benefit from efficient sequence modeling for long-range dependency capture.

Overall, graph-based representations model event streams as sparse relational structures rather than regular grids. They are effective for capturing local interactions and irregular spatio-temporal dependencies. However, their performance depends heavily on node construction, edge definition, and scalable graph updating under high event rates.

V-C Point-based Representation

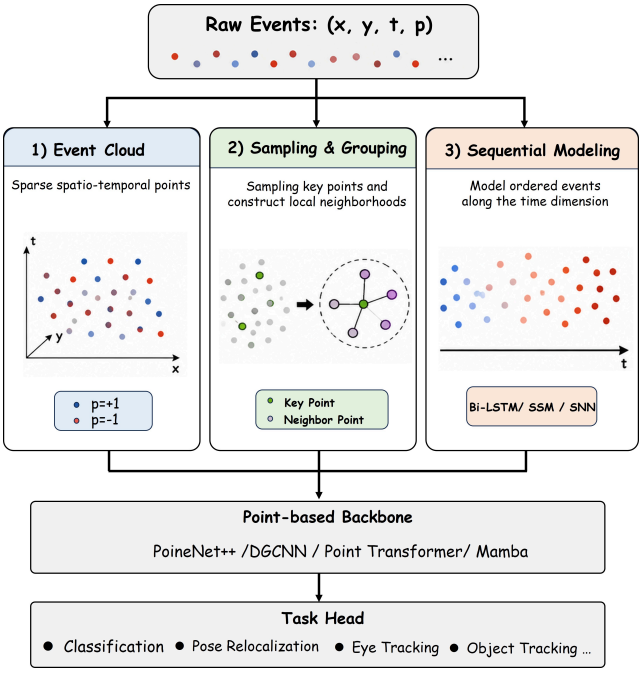


Fig. 9: Overview of point-based event representations. Point-based methods directly process sparse event clouds without converting events into dense grid structures. Typical pipelines include event cloud construction, local sampling and grouping, sequential temporal modeling, and point-based backbones for downstream event-based vision tasks.

The point-based representation treats an event stream as an unordered set of sparse spatio-temporal points,

$$\mathcal{P} = \{ p_1, p_2, ..., p_N \},$$

where each point p_i typically consists of spatial coordinates, timestamps, and polarity information. Unlike dense representations that require explicit projection onto regular grids, point-based representations operate directly on raw event streams without converting them into image-like or volume-like structures. By preserving the native sparsity and temporal precision of asynchronous events, they provide a natural framework for modeling fine-grained event dynamics directly from raw measurements.

A general point-based formulation can be written as

$$\mathbf{F} = \gamma\left(\underset{p_i \in \mathcal{P}}{\mathcal{A}} \left\{ h(p_i) \right\} \right), \quad (22)$$

where $h(\cdot)$ denotes an event-wise feature embedding function, $\mathcal{A}(\cdot)$ represents a feature aggregation operation over the event set, and $\gamma(\cdot)$ denotes a task-specific mapping function. Different point-based representations mainly differ in how local neighborhoods are constructed, how temporal dependencies are modeled, and how aggregation is performed over large event sets.

Early point-based representations primarily focused on exploiting the geometric structure of event clouds. Inspired by point cloud learning frameworks such as PointNet *[64]*, PointNet++ *[65]*, and DGCNN *[66]*, these methods regard events as sparse samples distributed in the spatio-temporal domain. ST-EC *[67]* introduced Space-time Event Clouds by treating timestamps as geometric coordinates and learning motion patterns directly from normalized event point distributions. PAT *[68]* further incorporates Gumbel Subset Sampling and self-attention to select representative event subsets while reducing computational complexity, enabling more effective modeling of long-range spatio-temporal interactions. EventNet *[69]* adopts recursive event-wise processing with temporal coding and hardware-friendly lookup tables, achieving real-time inference at event rates up to 1 MEPS. These studies establish the basic point-based paradigm in which event relationships are inferred through sparse neighborhood interactions rather than dense spatial layouts.

However, most early point-based methods primarily emphasize geometric feature extraction while underexploring the temporal dependencies embedded in asynchronous event streams. PEPNet *[70]* combines hierarchical point-wise features with attentive Bi-LSTM aggregation to jointly model spatial and temporal cues by explicitly exploiting the sequential nature of event timestamps. EventMamba *[71]* further integrates State Space Models into event cloud processing, enabling efficient modeling of long-range dependencies in chronologically ordered event sequences. These developments reflect an important transition from purely geometric reasoning toward explicit temporal modeling in point-based representations.

Recent studies increasingly focus on improving the efficiency and scalability of point-based representations. TTPOINT *[72]* compresses point feature extractors through Tensor Train Decomposition to achieve a favorable trade-off between recognition accuracy and computational efficiency. SpikePoint *[73]* combines point representations with spike-based computation to enable ultra-low-power event processing without frame conversion. E2B *[74]* demonstrates the effectiveness of raw event clouds for efficient object tracking through direct event-to-box prediction, while SECNet *[75]* introduces polarity-aware grouping together with Fourier-based feature extraction to improve scalability under high-resolution and long-duration event streams. These methods indicate a growing trend toward lightweight deployment and efficient processing of increasingly large event sets.

Overall, point-based representations preserve raw events as sparse spatio-temporal point sets without explicit frame or voxel conversion. They retain native sparsity and temporal precision, making them suitable for event-native modeling. However, they remain sensitive to sampling strategy, neighborhood construction cost, and limited spatial regularity.

VI From Representation Design to Representation Optimization

Recent studies suggest a gradual shift in event representation research: rather than continually proposing new encoding paradigms, increasing attention has been devoted to optimizing existing representations through data-driven objectives. These approaches seek to improve representation quality, robustness, and generalization without fundamentally altering the underlying representation form.

Representative directions include automated representation search, self-supervised representation learning, semantic decoupling, and multi-view consistency optimization. CHAOS *[53]* formulates representation selection as an automated search problem and employs the Gromov–Wasserstein discrepancy to identify suitable representation configurations without repeated downstream training. EvRepSL *[76]* exploits the physical consistency between events and intensity changes to learn noise-resilient representations in a self-supervised manner. Motion-Appearance Decoupling *[77]* explicitly separates static structures from motion dynamics to alleviate feature entanglement, while EventDance *[78]* leverages consistency across complementary representations to improve robustness and generalization.

These studies indicate that future advances may rely less on inventing entirely new representations and more on automatically optimizing, regularizing, and combining existing ones. This transition from *representation design* to *representation optimization* may offer a more scalable route toward robust and adaptive event-based perception.

VII Comparative Analysis of Empirical Results

To connect the representation taxonomy with empirical evidence, Table I summarizes representative results across different event-based vision tasks. These comparisons should not be interpreted as a strict leaderboard, since performance is affected not only by the representation itself, but also by backbone capacity, input modality, temporal windowing, training protocol, supervision, and dataset-specific settings. Nevertheless, several general observations can be drawn from the reported results.

First, representations that preserve stronger temporal fidelity, such as point-based, token-based, graph-based, or bit-level temporal encodings, often show advantages in scenarios where fine-grained motion dynamics and event ordering are important. Their strength comes from reducing the temporal collapse introduced by single-frame accumulation, although this usually increases the cost of sequence modeling, neighborhood construction, or attention computation. Second, dense representations, especially voxel-grid and related frame-like encodings, remain widely adopted when spatial alignment and pixel-level prediction are required. Their regular image-plane topology makes them compatible with mature convolutional, recurrent, and Transformer-based backbones, but this compatibility is obtained at the cost of temporal quantization and increased memory consumption. Third, compact representations such as event frames and binary maps remain valuable under latency, bandwidth, or hardware constraints, because they provide simple and efficient inputs even though they sacrifice event multiplicity or precise timestamps.

Overall, the empirical results suggest a representation–task trade-off rather than a universally dominant representation. Sparse and tokenized representations better preserve event-native temporal information, voxel-like representations provide stronger spatial regularity for dense prediction, and compact frame or binary representations offer higher efficiency. Future evaluations should compare representations under controlled backbones, temporal windows, input modalities, and computational budgets.

VIII Conclusion

This survey reviews event camera representation learning from the perspective of how asynchronous event streams are transformed into learnable structures. By organizing existing methods into dense-based and sparse-based paradigms, we summarize the design principles and trade-offs of event frames, time surfaces, binary maps, voxel grids, learning-based encodings, token-based representations, graph structures, and point-based event clouds. Our analysis shows that no single representation is universally optimal: dense representations provide spatial regularity and strong compatibility with mature neural architectures, but often sacrifice fine-grained temporal fidelity; sparse representations better preserve event-level timing and native sparsity, but require specialized modeling and may introduce additional computational costs. Therefore, the choice of representation should be understood as a balance among temporal precision, spatial topology, sparsity, efficiency, and architectural compatibility. Future research is expected to move beyond fixed handcrafted encodings toward adaptive, learnable, scalable, and hardware-aware representations that can preserve richer temporal information while remaining robust and efficient in real-world event-based vision systems.

TABLE I: Comprehensive comparison of event-based vision tasks. Methods are grouped by representation type within each dataset, and the best overall result within each dataset is highlighted.

Specific Task	Dataset	Method	Rep. Type	Metric	Performance
Action Recognition	DVS128 Gesture[11]	Deep SNN [11]	Event Frame	Acc. (%) ↑	91.80
		SLAYER [12]	Event Frame	Acc. (%) ↑	93.64
		TA-SNN [79]	Event Frame	Acc. (%) ↑	98.61
		STSC-SNN [80]	Event Frame	Acc. (%) ↑	98.96
		TCJA-SNN [81]	Event Frame	Acc. (%) ↑	99.00
		TBR [30]	Binary Map	Acc. (%) ↑	99.58
		VMST-Net [34]	Voxel Grid	Acc. (%) ↑	97.80
		EVSTr [35]	Voxel Grid	Acc. (%) ↑	98.60
		ACE-BET [82]	Voxel Grid	Acc. (%) ↑	98.88
		Motion-SNN [46]	Learning-based	Acc. (%) ↑	92.70
		GET [56]	Token-based	Acc. (%) ↑	97.90
		RG-CNN [58]	Graph-based	Acc. (%) ↑	97.20
		EDGCN+CRD [83]	Graph-based	Acc. (%) ↑	98.30
		ST-EC [67]	Point-based	Acc. (%) ↑	97.08
		SpikePoint [73]	Point-based	Acc. (%) ↑	98.74
		TTPOINT [72]	Point-based	Acc. (%) ↑	98.80
		SECNet [75]	Point-based	Acc. (%) ↑	98.90
		EventMamba [71]	Point-based	Acc. (%) ↑	99.20
	DailyDVS[46]	TimeSformer [84]	Event Frame	Acc. (%) ↑	90.60
		HMAX-SNN [85]	Time Surface	Acc. (%) ↑	68.30
		HMAX-SPA [86]	Time Surface	Acc. (%) ↑	76.90
		EVSTr [35]	Voxel Grid	Acc. (%) ↑	99.60
		Motion-SNN [46]	Learning-based	Acc. (%) ↑	90.30
		VMV-GCN [60]	Graph-based	Acc. (%) ↑	94.10
		SpikePoint [73]	Point-based	Acc. (%) ↑	97.92

Specific Task	Dataset	Method	Rep. Type	Metric	Performance
		EventMamba [71]	Point-based	Acc. (%) ↑	99.10
		TTPOINT [72]	Point-based	Acc. (%) ↑	99.10
		SECNet [75]	Point-based	Acc. (%) ↑	99.65
	DailyDVS-200[87]	EST [44]	Learning-based	Top-1 Acc. (%) ↑	32.23
		ESTF [88]	Token-based	Top-1 Acc. (%) ↑	24.68
		SDT [89]	Token-based	Top-1 Acc. (%) ↑	35.43
		Spikformer [90]	Token-based	Top-1 Acc. (%) ↑	36.94
		GET [56]	Token-based	Top-1 Acc. (%) ↑	37.28
Classification	N-MNIST[91]	SNN-BP [92]	Event Frame	Acc. (%) ↑	98.74
		EFV [93]	Graph-based	Acc. (%) ↑	98.90
		RG-CNN [58]	Graph-based	Acc. (%) ↑	99.00
		HATS [24]	Time Surface	Acc. (%) ↑	99.10
		ECSNet [94]	Point-based	Acc. (%) ↑	99.20
		Bina-Rep [31]	Binary Map	Acc. (%) ↑	99.34
		VMST-Net [34]	Voxel Grid	Acc. (%) ↑	99.50
		CTN [95]	Voxel Grid	Acc. (%) ↑	99.70
		GET [56]	Token-based	Acc. (%) ↑	99.70
		SECNet [75]	Point-based	Acc. (%) ↑	99.70
		EventTransformer [57]	Token-based	Acc. (%) ↑	99.90
	N-Caltech101[91]	HATS [24]	Time Surface	Acc. (%) ↑	64.20
		EVSTr [35]	Voxel Grid	Acc. (%) ↑	79.70
		CES [96]	Voxel Grid	Acc. (%) ↑	84.70
		E2VID [33]	Voxel Grid	Acc. (%) ↑	86.60
		ACE-BET [82]	Voxel Grid	Acc. (%) ↑	89.95
		EST [4]	Learning-based	Acc. (%) ↑	81.70
		Matrix-LSTM [45]	Learning-based	Acc. (%) ↑	85.72
		EventTransformer [57]	Token-based	Acc. (%) ↑	81.60
		RG-CNN [58]	Graph-based	Acc. (%) ↑	65.70
		AEGNN [62]	Graph-based	Acc. (%) ↑	66.80
		SlideGCN [61]	Graph-based	Acc. (%) ↑	76.10
		EDGCN+CRD [83]	Graph-based	Acc. (%) ↑	83.50
		SECNet [75]	Point-based	Acc. (%) ↑	82.40
Object Tracking	FE240hz	MambaEVT [13]	Event Frame	SR/PR ↑	58.09/91.97
		HDETrack [97]	Event Frame	SR/PR ↑	59.80/92.20
		HDETrack V2 [14]	Event Frame	SR/PR ↑	60.40/93.20
		STNet [47]	Learning-based	SR/PR ↑	58.50/89.60
		SFTrack-Slow [98]	Graph-based	SR/PR ↑	59.90/92.70
	VisEvent [99]	MambaEVT [13]	Event Frame	SR/PR/NPR ↑	35.90/50.20/39.40
		MambaEVT-P [13]	Event Frame	SR/PR/NPR ↑	37.20/51.80/40.80